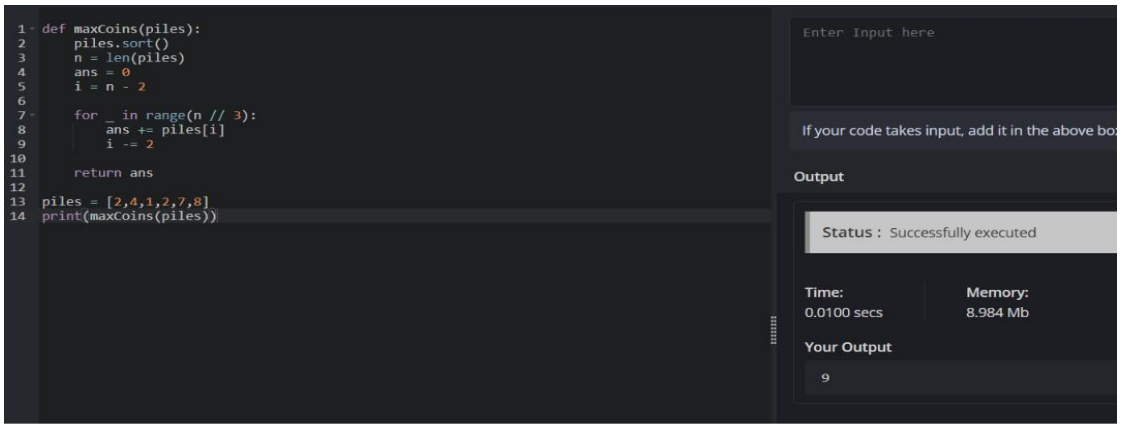
UNIT – 5 LAB EXERSICE

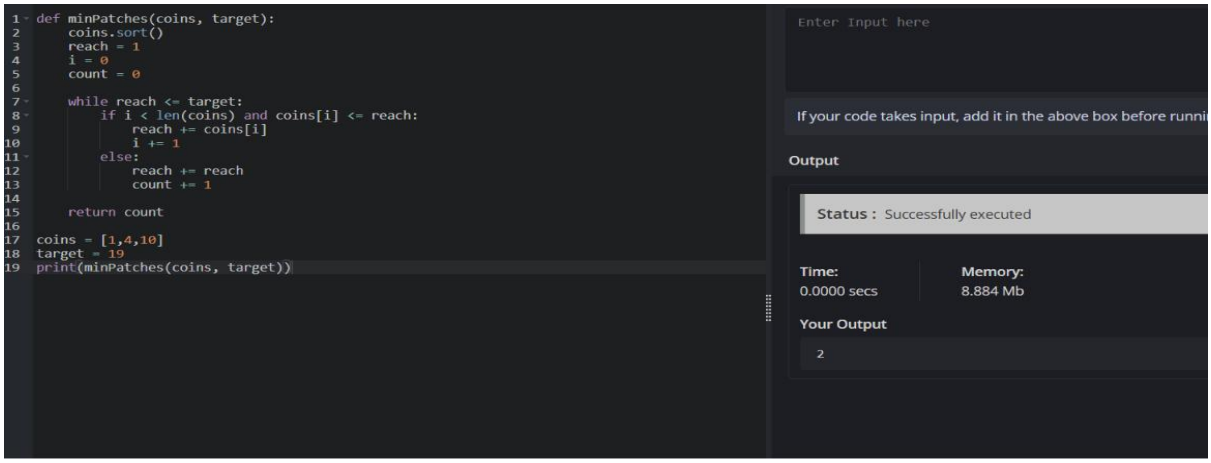
NAME : Jeeva

REG NO : 192421334

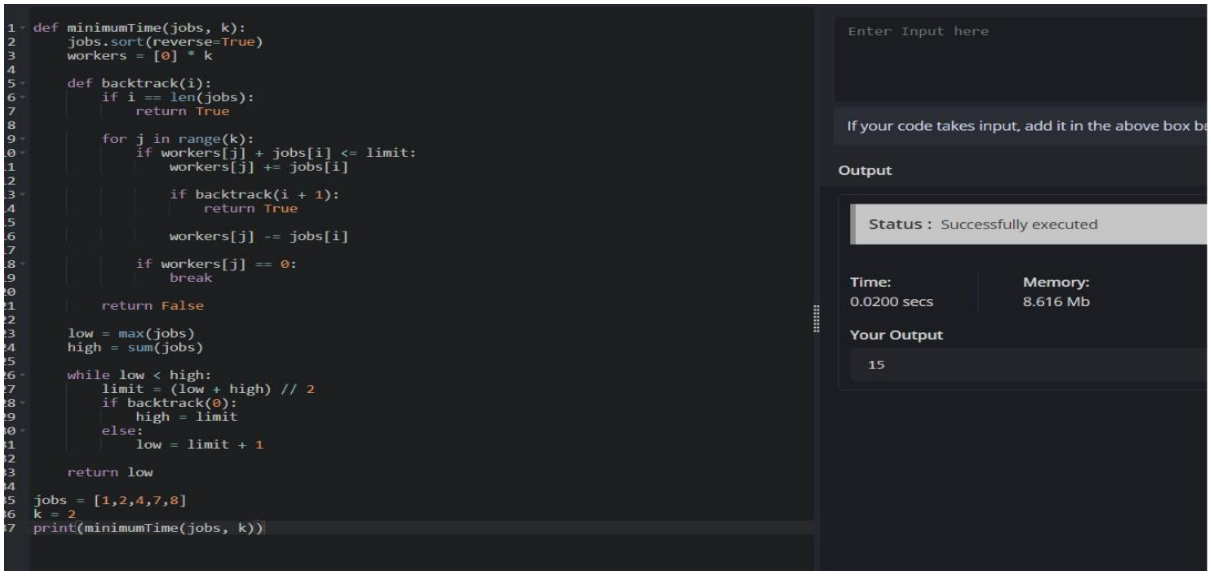
1. There are 3n piles of coins of varying size, you and your friends will take piles of coins as follows: In each step, you will choose any 3 piles of coins (not necessarily consecutive). Of your choice, Alice will pick the pile with the maximum number of coins. You will pick the next pile with the maximum number of coins. Your friend Bob will pick the last pile. Repeat until there are no more piles of coins. Given an array of integers piles where piles[i] is the number of coins in the ith pile. Return the maximum number of coins that you can have.



2. You are given a 0-indexed integer array coins, representing the values of the coins available, and an integer target. An integer x is obtainable if there exists a subsequence of coins that sums to x. Return the minimum number of coins of any value that need to be added to the array so that every integer in the range [1, target] is obtainable. A subsequence of an array is a new non- empty array that is formed from the original array by deleting some (possibly none) of the elements without disturbing the relative positions of the remaining elements.



3. You are given an integer array jobs, where jobs[i] is the amount of time it takes to complete the ith job. There are k workers that you can assign jobs to. Each job should be assigned to exactly one worker. The working time of a worker is the sum of the time it takes to complete all jobs assigned to them. Your goal is to devise an optimal assignment such that the maximum working time of any worker is minimized. Return the minimum possible maximum working time of any assignment.



4. We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i], obtaining a profit of profit[i]. You're given the startTime, endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time X you will be able to start another job that starts at time X.

```python
def maxProfit(start, end, profit):
    jobs = sorted(zip(start, end, profit))
    n = len(jobs)
    dp = [0] * (n + 1)

    for i in range(1, n + 1):
        s, e, p = jobs[i - 1]

        j = i - 1
        while j > 0 and jobs[j - 1][1] > s:
            j -= 1

        dp[i] = max(dp[i - 1], p + dp[j])

    return dp[n]

start = [1,2,3,3]
end = [3,4,5,6]
profit = [50,10,40,70]

print(maxProfit(start, end, profit))
```

```
Enter Input here

If your code takes input, add it in the above box be

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.684 Mb

Your Output

120
```

5. Given a graph represented by an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in the graph. The graph is represented as an adjacency matrix where graph[i][j] denote the weight of the edge from vertex i to vertex j. If there is no edge between vertices i and j, the value is Infinity (or a very large number).

```python
def dijkstra(graph, source):
    n = len(graph)
    dist = [float('inf')] * n
    visited = [False] * n
    dist[source] = 0

    for _ in range(n):
        u = -1

        for i in range(n):
            if not visited[i] and (u == -1 or dist[i] < dist[u]):
                u = i

        visited[u] = True

        for v in range(n):
            if graph[u][v] != float('inf'):
                dist[v] = min(dist[v], dist[u] + graph[u][v])

    return dist

I = float('inf')

graph = [
    [0,10,3,I,I],
    [I,0,1,2,I],
    [I,4,0,8,2],
    [I,I,I,0,7],
    [I,I,I,9,0]
]

print(dijkstra(graph, 0))
```

```
Enter Input here

If your code takes input, add it in the above box befo

Output

Status : Successfully executed

Time:
0.0200 secs

Memory:
9 Mb

Your Output

[0, 7, 3, 9, 5]
```

6. Given a graph represented by an edge list, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to a target vertex. The graph is represented as a list of edges where each edge is a tuple (u, v, w) representing an edge from vertex u to vertex v with weight w.

```python
import heapq

def dijkstra(n, edges, source, target):
    graph = [[] for _ in range(n)]

    for u, v, w in edges:
        graph[u].append((v, w))

    dist = [float('inf')] * n
    dist[source] = 0
    pq = [(0, source)]

    while pq:
        d, u = heapq.heappop(pq)

        if u == target:
            return d

        if d > dist[u]:
            continue

        for v, w in graph[u]:
            nd = d + w

            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))

    return dist[target]

edges = [
    (0,1,7),(0,2,9),(0,5,14),
    (1,2,10),(1,3,15),(2,3,11),
    (2,5,2),(3,4,6),(4,5,9)
]

print(dijkstra(6, edges, 0, 4))
```

```
Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.144 Mb

Your Output

26
```

7. Given a set of characters and their corresponding frequencies, construct the Huffman Tree and generate the Huffman Codes for each character.

```python
import heapq

def huffman(chars, freq):
    heap = [[f, c, ""] for c, f in zip(chars, freq)]
    heapq.heapify(heap)

    while len(heap) > 1:
        a = heapq.heappop(heap)
        b = heapq.heappop(heap)

        for x in a[1:]:
            if isinstance(x, str):
                pass

        heapq.heappush(heap, [a[0] + b[0], a, b])

    codes = {}

    def generate(node, code=""):
        if isinstance(node[1], str):
            codes[node[1]] = code
        else:
            generate(node[1], code + "0")
            generate(node[2], code + "1")

    generate(heap[0])
    return codes

chars = ['a','b','c','d']
freq = [5,9,12,13]

print(huffman(chars, freq))
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.144 Mb

Your Output

{'a': '00', 'b': '01', 'c': '10', 'd': '11'}
```

8. Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.

```python
def decode(codes, encoded):
    result = ""
    temp = ""

    reverse = {v: k for k, v in codes.items()}

    for bit in encoded:
        temp += bit

        if temp in reverse:
            result += reverse[temp]
            temp = ""

    return result

codes = {
    'a': '110',
    'b': '10',
    'c': '0',
    'd': '111'
}

encoded = "1101100111110"

print(decode(codes, encoded))
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.964 Mb

Your Output

aacda
```

9. Given a list of item weights and the maximum capacity of a container, determine the maximum weight that can be loaded into the container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.

```python
def maxWeight(weights, capacity):
    weights.sort(reverse=True)
    total = 0

    for w in weights:
        if total + w <= capacity:
            total += w

    return total

weights = [10,20,30,40,50]
capacity = 60

print(maxWeight(weights, capacity))
```

```
Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:
0.0200 secs

Memory:
8.784 Mb

Your Output

60
```

10. Given a list of item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach. The greedy approach should prioritize loading items into the current container until it is full before moving to the next container.

```python
def containers(weights, capacity):
    count = 1
    current = 0

    for w in weights:
        if current + w <= capacity:
            current += w
        else:
            count += 1
            current = w

    return count

weights = [5,10,15,20,25,30,35]
capacity = 50

print(containers(weights, capacity))
```

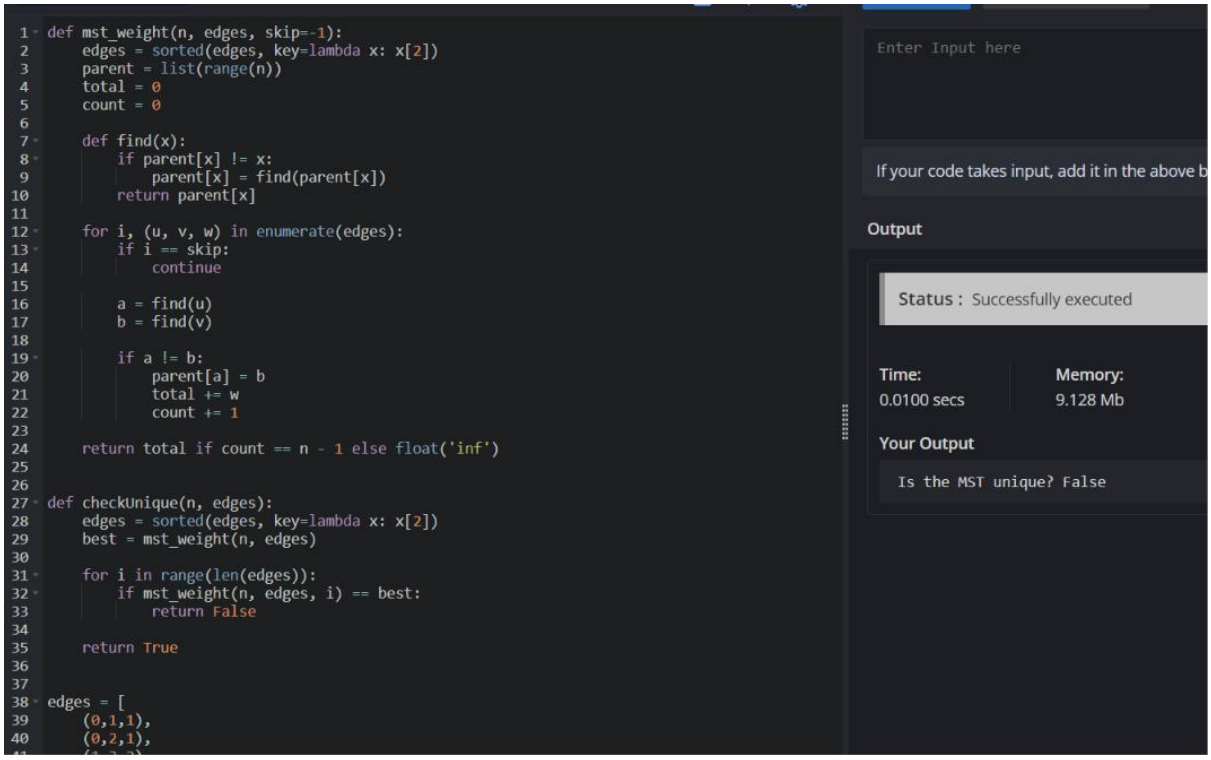
```
Enter Input here

If your code takes input, add it in the above box be

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.7 Mb

Your Output

4
```

11. Given a graph represented by an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) and its total weight.

```python
def kruskal(n, edges):
    edges.sort(key=lambda x: x[2])
    parent = list(range(n))
    mst = []
    total = 0

    def find(x):
        while parent[x] != x:
            x = parent[x]
        return x

    for u, v, w in edges:
        a = find(u)
        b = find(v)

        if a != b:
            parent[a] = b
            mst.append((u, v, w))
            total += w

            if len(mst) == n - 1:
                break

    return mst, total

edges = [
    (0,1,10),
    (0,2,6),
    (0,3,5),
    (1,3,15),
    (2,3,4)
]

mst, total = kruskal(4, edges)

print("Edges in MST:", mst)
print("Total weight:", total)
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.096 Mb

Your Output

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight: 19
```

12. Given a graph with weights and a potential Minimum Spanning Tree (MST), verify if the given MST is unique. If it is not unique, provide another possible MST.



```python
def mst_weight(n, edges, skip=-1):
    edges = sorted(edges, key=lambda x: x[2])
    parent = list(range(n))
    total = 0
    count = 0

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    for i, (u, v, w) in enumerate(edges):
        if i == skip:
            continue

        a = find(u)
        b = find(v)

        if a != b:
            parent[a] = b
            total += w
            count += 1

    return total if count == n - 1 else float('inf')


def checkUnique(n, edges):
    edges = sorted(edges, key=lambda x: x[2])
    best = mst_weight(n, edges)

    for i in range(len(edges)):
        if mst_weight(n, edges, i) == best:
            return False

    return True


edges = [
    (0,1,1),
    (0,2,1),
```

Enter Input here

If your code takes input, add it in the above b

Output

Status : Successfully executed

Time: 0.0100 secs

Memory: 9.128 Mb

Your Output

```
Is the MST unique? False
```